

Foundations of Artificial Intelligence

ADAPTIVE SUDOKU SOLVER

A PROJECT REPORT

Submitted by

2116240701558-TEJENDRA CHAPAI



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report “Adaptive Sudoku Solver” is the bonafide work of “Tejendra Chapai” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

ABSTRACT.....	2
1 INTRODUCTION.....	3
1.1 Background / Problem Context.....	4
1.2 Research Objectives.....	4
1.3 Scope	4
2 LITERATURE REVIEW	5
2.1 Existing Sudoku-Solving Approaches.....	5
2.2 Research Gap	6
3 PROPOSED SYSTEM.....	8
3.1 Architecture Diagram	9
4 MODULES DESCRIPTION	10
4.1 Module 1 — Sudoku Input and Validation ...	10
4.2 Module 2 — Puzzle Analysis and Computational Profiling.....	11
4.3 Module 3 — Adaptive Solving Engine.....	12
4.4 Module 4 — Solution Verification and Performance Evaluation	14
5 IMPLEMENTATION AND RESULTS.....	16
5.1 Experimental Setup	16
5.2 Results.....	18
6 CONCLUSION AND FUTURE WORK.....	25
7 REFERENCES.....	27

Abstract

Sudoku is a constraint-based puzzle that requires satisfying row, column, and subgrid constraints. Conventional Sudoku solvers generally use a predetermined technique such as backtracking, constraint propagation, or local search. However, Sudoku puzzles differ in their structural characteristics and computational difficulty, making a single strategy inefficient for all instances. This project proposes an **Adaptive Sudoku Solver** that dynamically selects an appropriate solving strategy based on puzzle characteristics and solving progress.

The system models Sudoku as a **Constraint Satisfaction Problem (CSP)** and integrates **constraint propagation, Minimum Remaining Values (MRV) with backtracking, and Min-Conflicts local search**. It analyzes features such as empty cells, candidate distribution, forced moves, and search complexity to select a suitable strategy. The solver can also switch strategies when the current approach fails to make sufficient progress.

The adaptive approach is evaluated against fixed solving strategies using metrics including **solving time, success rate, search nodes, backtracking operations, conflicts, and strategy switches**. The objective is to investigate whether adaptive strategy selection can improve solving efficiency across Sudoku puzzles with different structural characteristics.

CHAPTER 1

INTRODUCTION

1.1 Background / Problem Context

Sudoku is a logic-based number puzzle consisting of a 9×9 grid in which the digits 1 to 9 must be placed such that each digit appears exactly once in every row, column, and 3×3 subgrid. Due to its constraint-based structure, Sudoku can be formulated as a **Constraint Satisfaction Problem (CSP)** and used to study Artificial Intelligence techniques such as constraint propagation, heuristic search, backtracking, and local search.

Several computational techniques, including **backtracking**, **constraint propagation**, **Minimum Remaining Values (MRV)**, and **Min-Conflicts**, can be used to solve Sudoku puzzles. However, different puzzles may have different structural characteristics, such as the number of empty cells, candidate values, forced moves, and search requirements. Consequently, a single fixed solving strategy may not provide the same level of efficiency for every puzzle.

This project proposes an **Adaptive Sudoku Solver** that analyzes the characteristics of an input puzzle and selects an appropriate solving strategy from multiple established AI techniques. The system can also monitor solving progress and switch strategies when the current approach does not provide sufficient progress.

1.2 Research Objectives

1. To model Sudoku as a Constraint Satisfaction Problem.
2. To implement multiple Sudoku-solving strategies.
3. To analyze puzzle characteristics.

4. To develop an adaptive strategy-selection mechanism.
5. To evaluate adaptive and fixed strategies using measurable metrics.

1.3 Scope

The scope of the project is limited to standard 9×9 Sudoku puzzles and focuses on the adaptive selection and combination of existing AI techniques rather than developing a fundamentally new solving algorithm. The system aims to investigate whether adapting the solving strategy according to puzzle characteristics and solving progress can improve the efficiency of Sudoku solving.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Sudoku-Solving Approaches

Sudoku has been widely studied as a **Constraint Satisfaction Problem (CSP)** because the rows, columns, and subgrids impose well-defined constraints on the possible values of each cell. Constraint-processing techniques have been applied to Sudoku to assist with solving and to study how constraint satisfaction can guide the solving process [3]. Several computational approaches have been explored, including **backtracking search, constraint propagation, heuristic search, local search, and exact-cover methods**.

A basic **backtracking algorithm** solves Sudoku by assigning possible values to empty cells and backtracking when a constraint is violated. Constraint propagation improves the solving process by reducing the possible values of variables based on existing assignments. Heuristics such as **Minimum Remaining Values (MRV)** can further improve backtracking by selecting the most constrained cell first, allowing contradictions to be detected earlier. These techniques are representative of systematic constraint-based approaches to Sudoku solving [3].

Local search techniques, such as **Min-Conflicts**, approach constraint satisfaction by modifying an existing assignment to reduce the number of constraint violations. The Min-Conflicts heuristic was introduced as a repair-based approach for constraint satisfaction and scheduling problems, where variable values are changed to minimize conflicts [2]. Its application to

Sudoku has also been investigated through local-search and hybrid approaches. Stuckey et al. proposed a hybrid Sudoku-solving approach that combines constraint programming with iterated local search and incorporates Min-Conflicts-based search [4]. These approaches demonstrate that local search can complement systematic constraint-based techniques, although their behavior can differ depending on the problem instance.

Another established approach is **Algorithm X with Dancing Links (DLX)**, which provides an efficient implementation for exact-cover problems. Knuth introduced Dancing Links as a technique for efficiently implementing exact-cover search and related combinatorial problems [1]. Sudoku can be represented as an exact-cover problem, allowing this approach to be applied as a specialized solving technique.

These studies demonstrate that different Sudoku-solving techniques have different computational characteristics. Constraint-processing methods can directly reduce the domains of possible values [3], while Min-Conflicts and other local-search approaches attempt to reduce constraint violations [2], [4]. Exact-cover methods provide another systematic representation of the Sudoku problem [1]. Therefore, the choice of solving technique can influence the computational behavior of a solver.

2.2 Research Gap

Existing research has investigated individual solving techniques as well as combinations of multiple techniques. For example, constraint programming has been used to support Sudoku solving [3], while hybrid approaches have combined constraint programming and local search [4]. However, these approaches do not directly address the specific framework proposed in this project: **selecting or switching between multiple established solving**

strategies according to measurable characteristics of the current Sudoku state and observed solving progress. Sudoku puzzles may differ in the number of empty cells, candidate distribution, forced moves, and search requirements. This provides an opportunity to investigate an **adaptive solving framework** that selects an appropriate strategy and changes the strategy when the current approach does not provide sufficient progress.

Existing Sudoku-solving research has established several effective techniques, including constraint processing, systematic search, local search, and exact-cover methods [1]–[4]. Previous research has also investigated hybrid approaches that combine different solving techniques, demonstrating that multiple methods can complement each other [4]. However, Sudoku puzzles can exhibit different structural characteristics and computational requirements, meaning that the behavior of a particular solving strategy may vary across puzzle instances and solving states.

While existing studies have explored individual algorithms, constraint-based approaches, and hybrid combinations, there is scope for investigating an **adaptive framework that explicitly analyzes measurable characteristics of the current puzzle state and uses them to select or switch between established solving strategies during execution.** The proposed **Adaptive Sudoku Solver** addresses this gap by combining **Constraint Propagation, MRV-based Backtracking, and Min-Conflicts** within an adaptive control framework. The system analyzes puzzle characteristics and solving progress to determine when a strategy should be selected or changed. Its performance is then compared with fixed solving strategies using measurable metrics such as **solving time, search nodes, backtracking operations, conflicts, and strategy switches.**

CHAPTER 3

PROPOSED SYSTEM

The proposed system is an **Adaptive Sudoku Solver** designed to solve standard 9×9 Sudoku puzzles by selecting and, when required, switching between multiple established Artificial Intelligence solving strategies. Instead of applying a single fixed algorithm to every puzzle, the system first analyzes the input puzzle and creates a computational profile based on measurable characteristics such as empty cells, candidate values, and forced moves. This profile is then used to select a suitable solving strategy.

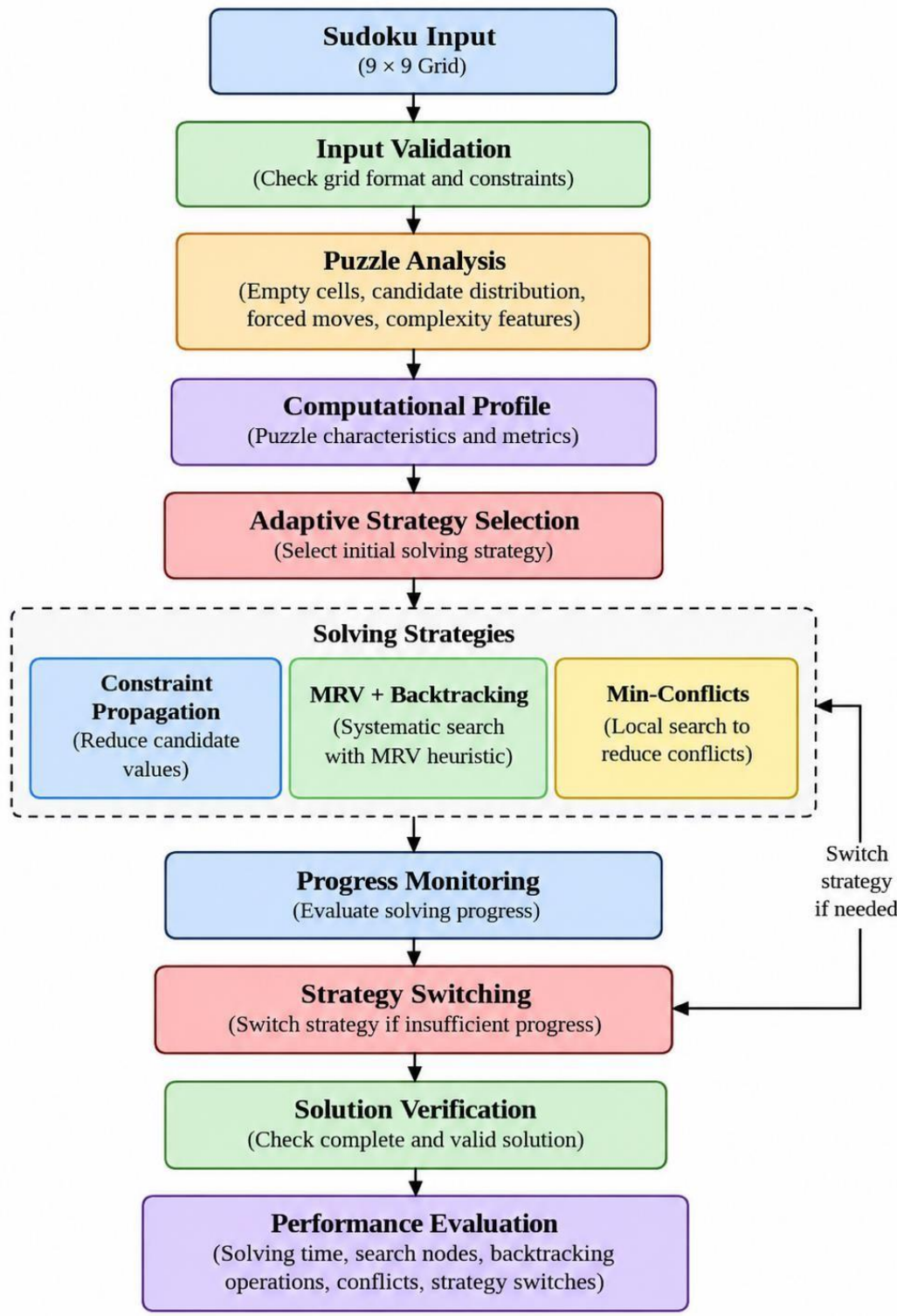
The system accepts a manually entered Sudoku grid as input. The input grid is converted into a digital 9×9 representation and validated before the solving process begins.

After validation, the **Puzzle Analysis** stage extracts relevant characteristics from the Sudoku grid. These characteristics are used by the **Adaptive Strategy Selection** component to choose among three primary solving approaches: **Constraint Propagation**, **MRV-based Backtracking**, and **Min-Conflicts Local Search**. Constraint propagation is used to eliminate invalid candidates and identify forced values. MRV-based backtracking selects the most constrained cell when systematic search is required. Min-Conflicts uses local search to reduce constraint violations.

During the solving process, the system monitors its progress using measures such as cells solved, candidate reduction, search activity, and conflicts. If the selected strategy fails to make sufficient progress, the system can switch to

another suitable strategy and continue solving. The overall workflow is given below.

3.1 ARCHITECTURE DIAGRAM



CHAPTER 4

MODULES DESCRIPTION

4.1 Module 1 — Sudoku Input and Validation

The **Sudoku Input and Validation Module** is the initial module of the proposed Adaptive Sudoku Solver. Its purpose is to accept a standard **9×9 Sudoku grid** and verify that the input is valid before the solving process begins. The input grid is represented internally as a 9×9 matrix, where digits from 1 to 9 represent filled cells and 0 represents empty cells.

After receiving the input, the validation component checks the basic Sudoku constraints. Each row, column, and 3×3 subgrid must not contain duplicate non-zero values. If a duplicate or invalid value is detected, the puzzle is considered invalid and the solving process is stopped. If the input satisfies the required constraints, it is passed to the Puzzle Analysis and Computational Profiling Module. This module forms the first stage of the proposed workflow.

The module performs the following operations:

1. Accept Sudoku Grid

Receives a standard 9×9 Sudoku puzzle as input.

2. Input Format Checking

Ensures that the input contains exactly 9 rows and 9 columns and that all values are within the permitted range.

3. Row Validation

Checks that no non-zero digit appears more than once in a row.

4. Column Validation

Checks that no non-zero digit appears more than once in a column.

5. 3×3 Subgrid Validation

Checks that no non-zero digit appears more than once within each 3×3 subgrid.

6. Validation Result

If the puzzle is valid, it is passed to the Puzzle Analysis and Computational Profiling Module. Otherwise, an appropriate invalid-input result is returned.

4.2 Module 2 — Puzzle Analysis and Computational Profiling

The **Puzzle Analysis and Computational Profiling Module** analyzes a validated Sudoku puzzle to determine its structural and computational characteristics. The information obtained from this module is used by the Adaptive Strategy Selection component to select an appropriate solving strategy.

After receiving a validated 9×9 Sudoku grid from Module 1, the module identifies the empty cells and calculates the possible candidate values for each empty cell based on the existing row, column, and 3×3 subgrid constraints.

The module analyzes the following characteristics:

1. Empty Cells

Counts the number of unfilled cells in the Sudoku grid.

2. Candidate Values

Determines the possible values that can be placed in each empty cell.

3. Candidate Distribution

Examines the number of candidates available across the empty cells to determine how constrained the puzzle is.

4. Forced Moves

Identifies cells having only one possible candidate, allowing those cells to be filled directly.

5. Search Complexity

Estimates the amount of search that may be required based on the available candidates and constraints.

6. Computational Profile

Combines the extracted characteristics into a computational profile. This profile is passed to the Adaptive Strategy Selection component for selecting an appropriate solving strategy.

4.3 Module 3 — Adaptive Solving Engine

The **Adaptive Solving Engine** is the core module of the proposed Adaptive Sudoku Solver. It uses the computational profile generated by Module 2 to select an appropriate solving strategy. Instead of applying one fixed algorithm to every puzzle, the module can select and switch between multiple established solving techniques based on the characteristics of the puzzle and the progress of the solving process.

The module incorporates three primary solving strategies:

1. Constraint Propagation

Constraint Propagation reduces the candidate values of empty cells using the Sudoku constraints. When a cell has only one valid candidate, that value can be assigned directly. This approach is particularly useful when the puzzle contains a sufficient number of forced moves.

2. MRV + Backtracking

The **Minimum Remaining Values (MRV)** heuristic selects the empty cell having the smallest number of possible candidates. The selected value is then explored using systematic backtracking. If a contradiction occurs, the solver backtracks and tries another candidate. This approach provides systematic search when direct constraint propagation is insufficient.

3. Min-Conflicts

The **Min-Conflicts** strategy uses local search to reduce the number of constraint violations in a candidate Sudoku assignment. Values are modified in cells contributing to conflicts in an attempt to reach a valid solution.

Adaptive Strategy Selection

The Adaptive Solving Engine initially selects a strategy using the computational profile generated by Module 2. The selection is based on characteristics such as the number of empty cells, candidate distribution, forced moves, and estimated search requirements.

Progress Monitoring and Strategy Switching

During solving, the system monitors the progress of the selected strategy using indicators such as cells solved, candidate reduction, search activity, and conflicts. If sufficient progress is being made, the solver continues using

the current strategy. If progress becomes insufficient, the adaptive controller can switch to another available strategy and continue the solving process.

Thus, the Adaptive Solving Engine provides the central mechanism through which the proposed system differs from a fixed-strategy Sudoku solver.

4.4 Module 4 — Solution Verification and Performance Evaluation

The **Solution Verification and Performance Evaluation Module** is responsible for confirming the correctness of the generated Sudoku solution and measuring the performance of the different solving strategies. After a solution is produced by Module 3, the verification component checks whether all Sudoku constraints are satisfied. Each row, column, and 3×3 subgrid must contain the digits 1 to 9 without repetition.

The module also records performance information generated during the solving process. These measurements are used to evaluate the Adaptive Sudoku Solver against the individual fixed strategies.

The module performs the following operations:

1. Solution Verification

Checks whether the completed grid satisfies all Sudoku constraints.

2. Solving Time Measurement

Records the time required to solve the puzzle.

3. Search Node Measurement

Records the number of search states or nodes explored during systematic search.

4. Backtracking Measurement

Counts the number of backtracking operations performed.

5. Conflict Measurement

Records the number of conflicts generated during local-search solving.

6. Strategy Switch Measurement

Records the number of times the adaptive solver changes from one solving strategy to another.

7. Performance Comparison

Compares the Adaptive Sudoku Solver with the individual fixed strategies using the collected performance metrics.

The collected measurements are subsequently used in the experimental evaluation to compare **Constraint Propagation, MRV + Backtracking, Min-Conflicts, and the Adaptive Sudoku Solver.**

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

Hardware and Software Configuration

The Adaptive Sudoku Solver was implemented and tested on a personal computer using a Python-based development environment. The implementation was developed using Visual Studio Code as the integrated development environment and Python 3.13.14 as the programming language. A Python virtual environment was used to isolate the project dependencies and maintain a consistent execution environment.

The project uses standard Python libraries along with NumPy, Pandas, and Matplotlib for numerical processing, result management, and visualization. Git was used for version control, while GitHub was used to maintain the project repository and its development history.

Input Dataset

The experimental evaluation was performed using a benchmark set of eight standard 9×9 Sudoku puzzles. The puzzles were prepared with different numbers of empty cells to provide variation in their computational characteristics. The benchmark contains puzzles with 20, 30, 40, 50, 55, 60, 65, and 50 empty cells respectively.

Each puzzle was represented digitally as a 9×9 matrix, where the values 1 to 9 represent filled cells and 0 represents an empty cell. Before solving, each

puzzle was checked for valid dimensions, valid values, and consistency of the given values across rows, columns, and 3×3 subgrids.

Algorithms Evaluated

Four solver configurations were considered during the experimental evaluation:

1. **Constraint Propagation** – repeatedly assigns values to cells having a single valid candidate.
2. **MRV + Backtracking** – uses the Minimum Remaining Values heuristic to select a constrained cell and applies recursive backtracking.
3. **Min-Conflicts** – uses local search to reduce conflicts in a candidate Sudoku assignment.
4. **Adaptive Sudoku Solver** – analyzes the computational characteristics of the current puzzle and selects or switches between constraint propagation, MRV-based backtracking, and Min-Conflicts.

The adaptive solver records the strategies used during execution and can switch strategies when the current approach does not provide sufficient progress.

Performance Metrics

The performance of the different solver configurations was evaluated using several measurable metrics. These include:

- **Solving Time:** Time required to process a Sudoku puzzle.
- **Success Rate:** Percentage of benchmark puzzles successfully solved.
- **Search Nodes:** Number of search states explored during solving.
- **Backtracking Operations:** Number of times the solver had to undo a previous search decision.

- **Conflicts:** Number of constraint conflicts encountered during local search.
- **Strategy Switches:** Number of times the adaptive solver changed from one solving strategy to another.

These metrics allow the adaptive approach to be evaluated not only by whether a puzzle is solved, but also by the computational behavior of the solver.

Experimental Procedure

Each benchmark puzzle was supplied separately to the four solver configurations. The solvers were executed using the same set of benchmark puzzles, and their performance measurements were recorded automatically.

The fixed strategies were evaluated independently to establish baseline performance. The Adaptive Sudoku Solver first analyzed each puzzle, selected a strategy based on the computational profile, and monitored solving progress. When sufficient progress was not obtained, the controller could select another available strategy.

The recorded measurements were stored for subsequent analysis. The results were then processed to generate comparison tables and graphical visualizations showing solving time, success rate, search activity, and adaptive strategy switching behavior.

5.2 RESULTS

The proposed Adaptive Sudoku Solver was evaluated using eight standard 9×9 Sudoku puzzles containing different numbers of empty cells. The performance of the Adaptive Solver was compared with three fixed solving approaches: Constraint Propagation, MRV + Backtracking, and Min-

Conflicts. The evaluation considered solving time, success rate, search nodes, backtracking operations, conflicts, and strategy switches.

Table 5.1 Comparison of Solver Performance

Solver	Puzzles Solved	Success Rate (%)	Avg.Solving Time (s)	Avg.Search Nodes	Avg.Backtracks	Avg.Strategy Switches
Constraint Propagation	3/8	37.5	0.001053	0.00	0.00	0.00
MRV + Backtracking	8/8	100.0	0.017572	48.62	2.38	0.00
Min-Conflicts	0/8	0.0	0.647065	1000.0	0.00	0.00
Adaptive	8/8	100.0	0.027370	54.88	2.38	0.25

Result Analysis

The benchmark results show that **Constraint Propagation successfully solved 3 out of the 8 puzzles**, corresponding to a success rate of 37.5%. It was effective for puzzles where a large number of cells could be determined directly through constraint propagation. However, it could not complete the puzzles that required systematic search.

The **MRV + Backtracking solver successfully solved all eight puzzles**, achieving a 100% success rate. It required an average of 48.62 search nodes and 2.38 backtracking operations per puzzle. This demonstrates the effectiveness of the MRV heuristic in reducing unnecessary search by selecting the most constrained cell.

The **Min-Conflicts solver did not successfully solve any of the eight benchmark puzzles under the configured search budget**. It reached the maximum number of search steps for each puzzle and produced a relatively

high number of conflicts. Its average solving time was also higher than the other approaches. This indicates that the current Min-Conflicts configuration was not sufficient for reliably solving the benchmark set.

The **Adaptive Sudoku Solver successfully solved all eight puzzles**, achieving a 100% success rate. It selected different strategies according to the characteristics of the puzzles and, when necessary, switched strategies during solving. The adaptive approach required an average of 54.88 search nodes and 2.38 backtracking operations.

Although the Adaptive Solver achieved the same success rate as MRV + Backtracking, its average solving time was higher in this benchmark. This additional overhead is associated with puzzle analysis, strategy selection, progress monitoring, and strategy switching. Therefore, the current experimental results demonstrate the ability of the proposed system to perform adaptive strategy selection, but they do not establish that the adaptive approach is faster than the fixed MRV-based approach.

Table 5.2 Adaptive Strategy Selection and Switching

Puzzle	Empty Cells	Adaptive-Strategy Sequence	Switches	Solved
1	20	Constraint Propagation	0	es
2	30	Constraint Propagation	0	es
3	40	MRV + Backtracking	0	es

Puzzle	Empty Cells	Adaptive-Strategy Sequence	Switches	Solved
4	50	MRV + Backtracking	0	es
5	55	MRV + Backtracking	0	es
6	60	MRV + Backtracking	0	es
7	65	Min-Conflicts → Constraint Propagation → MRV + Backtracking	2	es
8	50	MRV + Backtracking	0	es

Adaptive Strategy Analysis

The adaptive strategy-selection results show that the controller selected **Constraint Propagation** for the smaller puzzles P1 and P2, where sufficient forced moves were available to solve the puzzles without search.

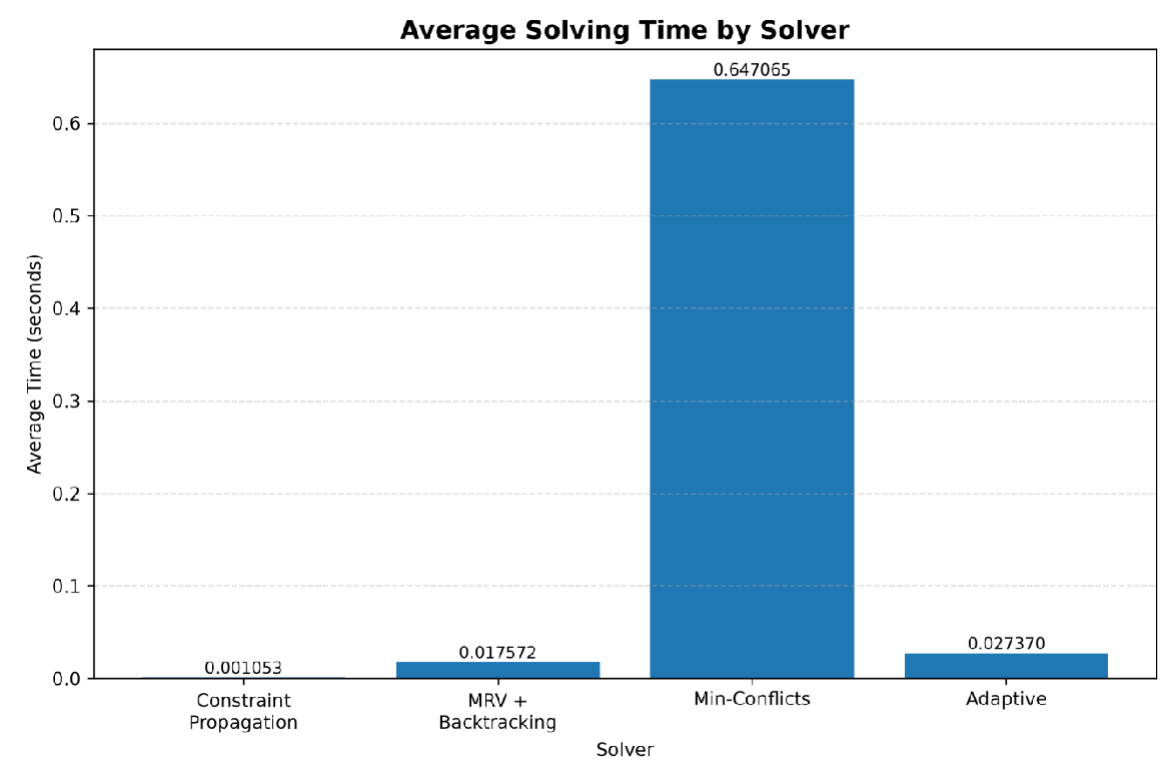
For P3 through P6 and P8, the adaptive controller selected **MRV + Backtracking**, which successfully solved these puzzles without requiring a strategy switch.

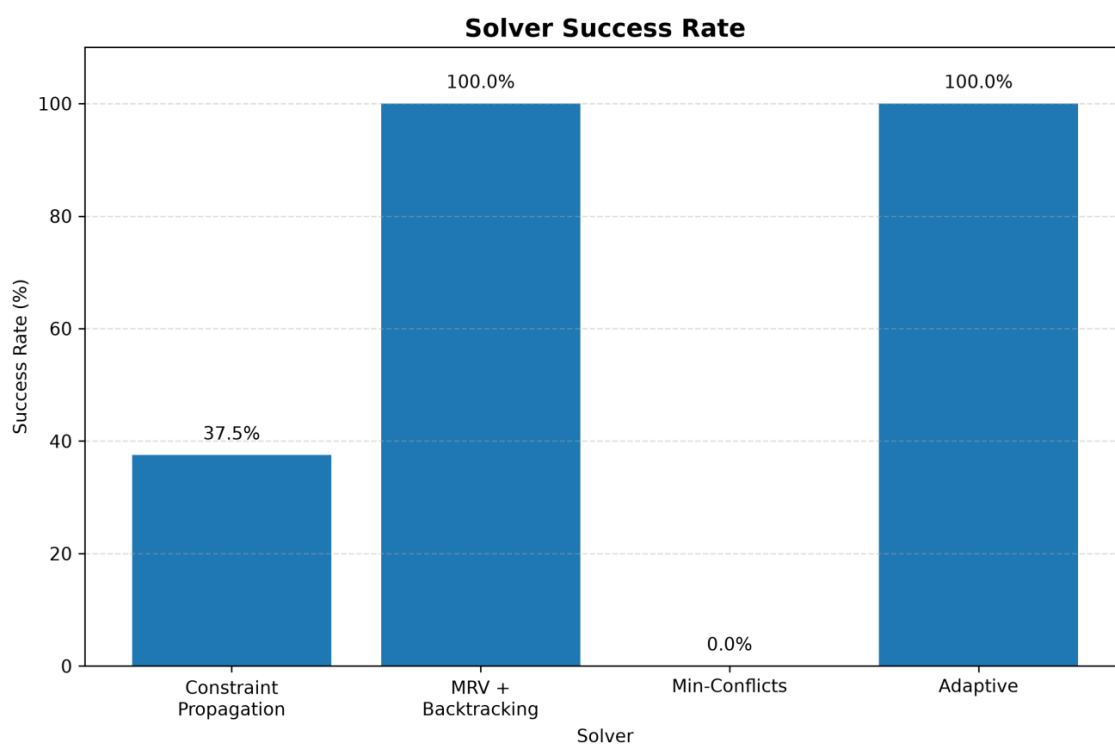
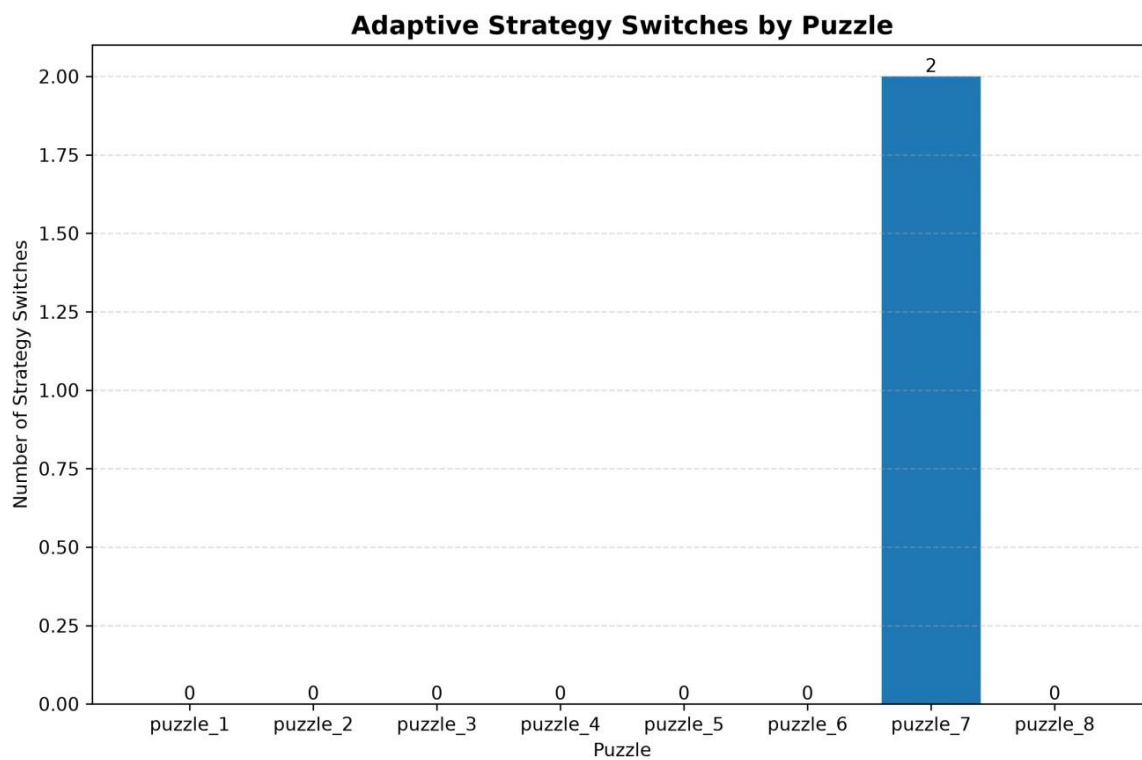
Puzzle P7 demonstrated the dynamic switching behavior of the proposed framework. The adaptive solver initially selected **Min-Conflicts**, but the strategy did not provide sufficient progress within its configured search

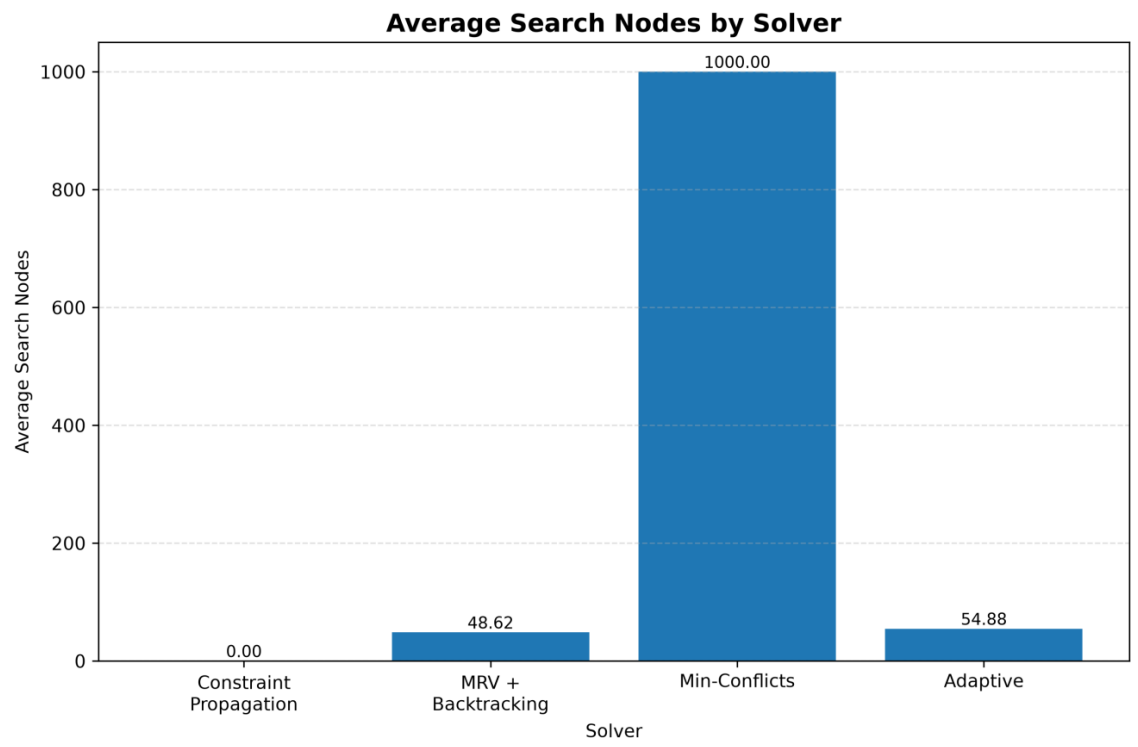
budget. The controller subsequently switched to Constraint Propagation and then to MRV + Backtracking, which completed the puzzle. Thus, P7 produced **two strategy switches**.

This result demonstrates the main mechanism proposed in the project: the solver is not restricted to a single solving strategy and can change its strategy when the current approach does not provide sufficient progress.

5.2.1 Performance Visualization







CHAPTER 6

CONCLUSION AND FUTURE WORK

Conclusion

The Adaptive Sudoku Solver was developed as an Artificial Intelligence-based framework for solving standard 9×9 Sudoku puzzles using multiple solving strategies. The system models Sudoku as a Constraint Satisfaction Problem and incorporates Constraint Propagation, Minimum Remaining Values (MRV) with Backtracking, and Min-Conflicts Local Search.

The proposed framework analyzes measurable characteristics of a Sudoku puzzle, including the number of empty cells, candidate distribution, forced moves, and computational requirements. Based on this computational profile, the adaptive controller selects an appropriate solving strategy and can switch to another strategy when sufficient progress is not obtained.

The experimental evaluation was conducted using eight benchmark Sudoku puzzles with different numbers of empty cells. Constraint Propagation solved 3 out of 8 puzzles, while MRV + Backtracking solved all 8 puzzles. Min-Conflicts did not solve any of the benchmark puzzles within the configured search budget. The Adaptive Sudoku Solver successfully solved all 8 puzzles and demonstrated actual strategy switching on one of the benchmark instances.

The results demonstrate that adaptive strategy selection can combine the strengths of different Sudoku-solving techniques within a single framework. However, the adaptive approach introduced additional computational overhead due to puzzle analysis, strategy selection, and progress monitoring. Therefore, the current results demonstrate the feasibility of adaptive solving rather than establishing that the adaptive approach is faster than every fixed strategy.

Future Work

The following improvements can be considered for future development:

1. **Improved Strategy Selection**

The strategy-selection mechanism can be refined using a larger benchmark dataset and more detailed puzzle characteristics.

2. **Additional Solving Algorithms**

Other techniques such as Algorithm X with Dancing Links (DLX), SAT solving, and advanced constraint-propagation methods can be incorporated into the adaptive framework.

3. **Machine Learning-Based Selection**

A machine-learning model could be trained using benchmark results

to predict which solving strategy is likely to perform effectively for a given Sudoku puzzle.

4. **Dynamic Strategy Switching**

The current switching mechanism can be improved to make decisions based on more detailed real-time indicators such as search stagnation, branching factor, candidate reduction, and conflict growth.

5. **Larger Benchmark Dataset**

Future experiments can use a substantially larger collection of Sudoku puzzles with different structural characteristics to provide more comprehensive performance evaluation.

6. **Support for Different Sudoku Variants**

The framework can be extended to Sudoku variants such as 16×16 Sudoku and other constraint-based puzzle formats.

7. **Image-Based Sudoku Recognition**

The system can be extended to support image-based Sudoku recognition. An image-processing pipeline could be developed to detect the Sudoku grid, segment individual cells, and recognize the digits before passing the extracted 9×9 grid to the adaptive solving framework.

CHAPTER 7

REFERENCES

D. E. Knuth, “Dancing Links,” arXiv:cs/0011047, 2000.[1]

S. Minton, M. D. Johnston, A. B. Philips, and P. Laird,
“Minimizing Conflicts: A Heuristic Repair Method for Constraint
Satisfaction and Scheduling Problems,” *Artificial Intelligence*,
vol. 58, nos. 1–3, pp. 161–205, 1992, doi:
10.1016/0004-3702(92)90007-K.[2]

C. G. Reeson, K.-C. Huang, K. M. Bayer, and B. Y. Choueiry,
“An Interactive Constraint-Based Approach to Sudoku,”
Proceedings of the AAAI Conference on Artificial Intelligence,
Vancouver, Canada, pp. 1976–1977, 2007.[3]

P. J. Stuckey, T. Feydy, A. Schutt, G. Tack, and J. Fischer,
“A Hybrid Approach for the Sudoku Problem: Using Constraint
Programming in Iterated Local Search,” *IEEE Intelligent Systems*,
vol. 32, no. 2, pp. 52–62, 2017, doi: 10.1109/MIS.2017.29.[4]

